Advanced Expression Manipulation

In this section, we discuss some ways that we can perform advanced manipulation of expressions.

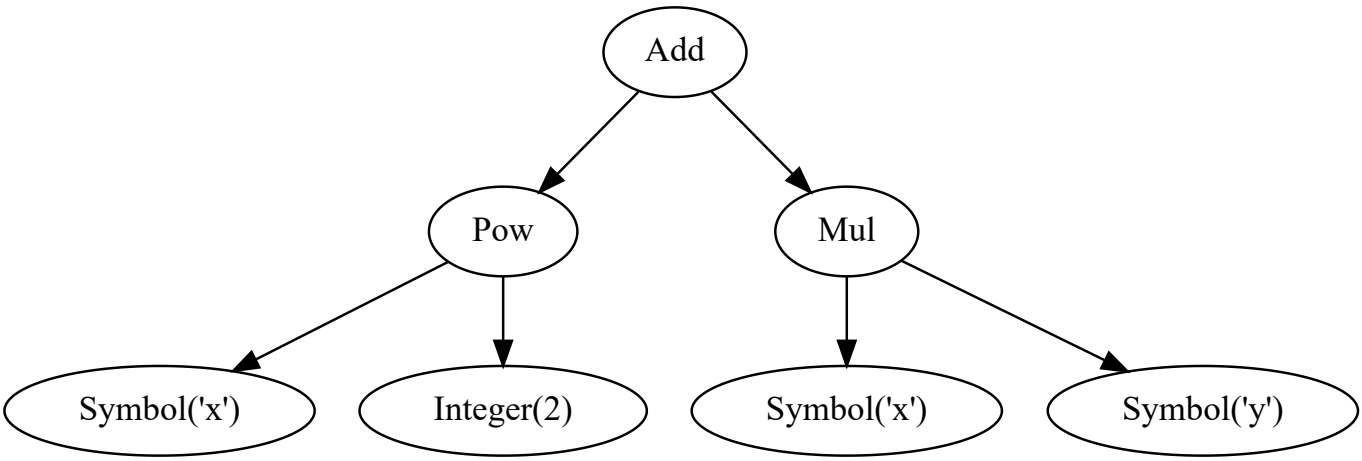
Understanding Expression Trees

Before we can do this, we need to understand how expressions are represented in SymPy. A mathematical expression is represented as a tree. Let us take the expression $x^2 + xy$, i.e., `x**2 + x*y`. We can see what this expression looks like internally by using `srepr`

```
>>> from sympy import *
>>> x, y, z = symbols('x y z')
```

```
>>> expr = x**2 + x*y
>>> srepr(expr)
"Add(Pow(Symbol('x'), Integer(2)), Mul(Symbol('x'), Symbol('y')))"
```

The easiest way to tear this apart is to look at a diagram of the expression tree:



> **Note**
>
> The above diagram was made using Graphviz and the `dotprint` function.

First, let's look at the leaves of this tree. Symbols are instances of the class Symbol. While we have been doing

```
>>> x = symbols('x')
```

''' '-o done

Skip to content

```
>>> x = Symbol('x')
```

Either way, we get a Symbol with the name "x" [1]. For the number in the expression, 2, we got `Integer(2)`. `Integer` is the SymPy class for integers. It is similar to the Python built-in type `int`, except that `Integer` plays nicely with other SymPy types.

When we write `x**2`, this creates a `Pow` object. `Pow` is short for "power".

```
>>> srepr(x**2)
"Pow(Symbol('x'), Integer(2))"
```

We could have created the same object by calling `Pow(x, 2)`

```
>>> Pow(x, 2)
x**2
```

Note that in the `srepr` output, we see `Integer(2)`, the SymPy version of integers, even though technically, we input `2`, a Python int. In general, whenever you combine a SymPy object with a non-SymPy object via some function or operation, the non-SymPy object will be converted into a SymPy object. The function that does this is `sympify` [2].

```
>>> type(2)
<... 'int'>
>>> type(sympify(2))
<class 'sympy.core.numbers.Integer'>
```

We have seen that `x**2` is represented as `Pow(x, 2)`. What about `x*y`? As we might expect, this is the multiplication of `x` and `y`. The SymPy class for multiplication is `Mul`.

```
>>> srepr(x*y)
"Mul(Symbol('x'), Symbol('y'))"
```

Thus, we could have created the same object by writing `Mul(x, y)`.

```
>>> Mul(x, y)
x*y
```

Now we get to our final expression, `x**2 + x*y`. This is the addition of our last two objects, `Pow(x, 2)`, and `Mul(x, y)`. The SymPy class for addition is `Add`, so, as you might expect, to create this object, we use `Add(Pow(x, 2), Mul(x, y))`.

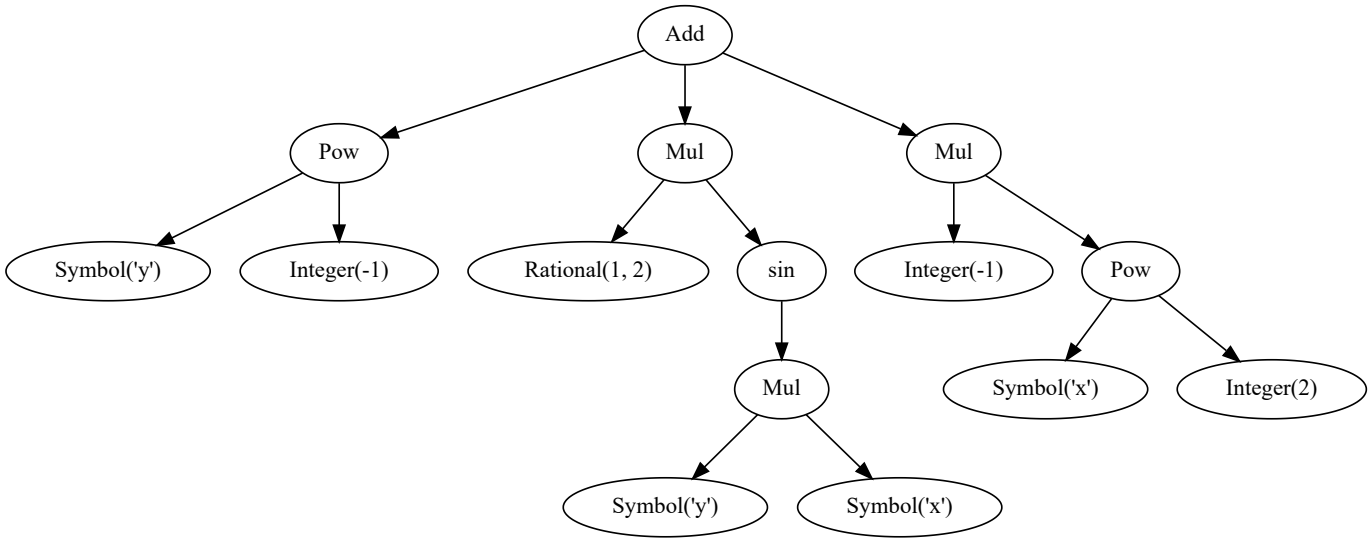
```
>>> Add(Pow(x, 2), Mul(x, y))
x**2 + x*y
```

SymPy expression trees can have many branches, and can be quite deep or quite broad. Here is a

Skip to content example

```
>>> expr = sin(x*y)/2 - x**2 + 1/y
>>> srepr(expr)
"Add(Mul(Integer(-1), Pow(Symbol('x'), Integer(2))), Mul(Rational(1, 2),
sin(Mul(Symbol('x'), Symbol('y')))), Pow(Symbol('y'), Integer(-1)))"
```

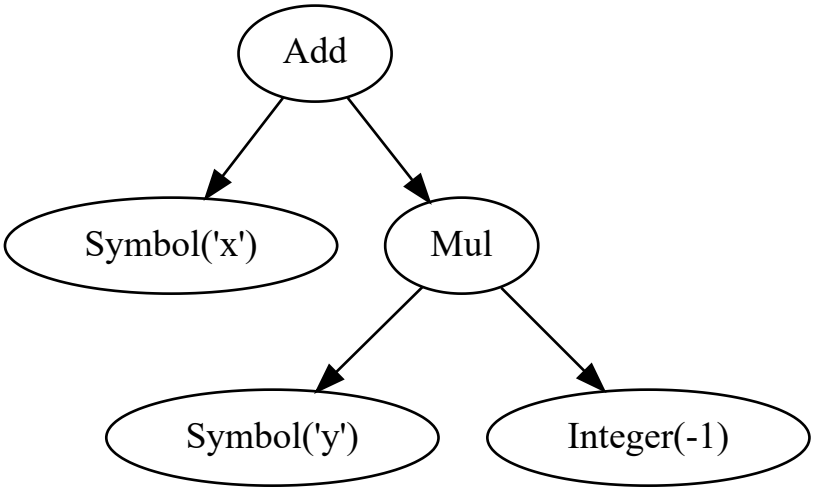
Here is a diagram



This expression reveals some interesting things about SymPy expression trees. Let's go through them one by one.

Let's first look at the term `x**2`. As we expected, we see `Pow(x, 2)`. One level up, we see we have `Mul(-1, Pow(x, 2))`. There is no subtraction class in SymPy. `x - y` is represented as `x + -y`, or, more completely, `x + -1*y`, i.e., `Add(x, Mul(-1, y))`.

```
>>> srepr(x - y)
"Add(Symbol('x'), Mul(Integer(-1), Symbol('y')))"
```

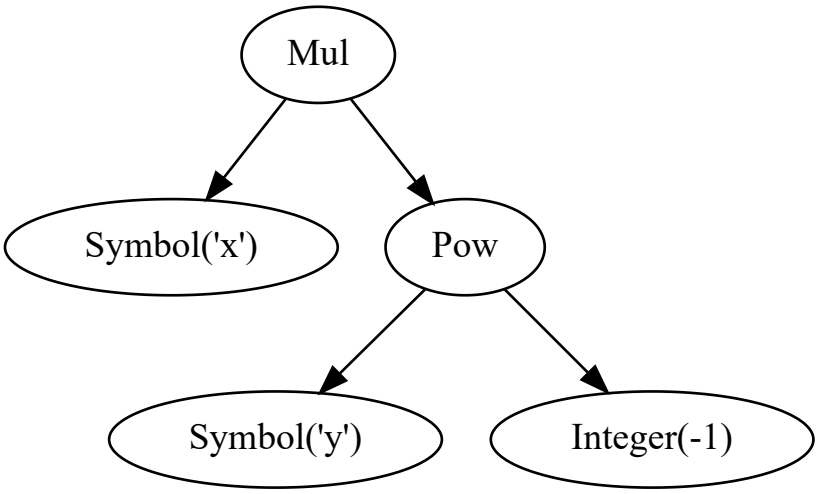


Next, look at `1/y`. We might expect to see something like `Div(1, y)`, but similar to subtraction, there is no class in SymPy for division. Rather, division is represented by a power of -1. Hence, we

Skip to content

What if we had divided something other than 1 by `y`, like `x/y`? Let's see.

```
>>> expr = x/y
>>> srepr(expr)
"Mul(Symbol('x'), Pow(Symbol('y'), Integer(-1)))"
```



We see that `x/y` is represented as `x*y**-1`, i.e., `Mul(x, Pow(y, -1))`.

Finally, let's look at the `sin(x*y)/2` term. Following the pattern of the previous example, we might expect to see `Mul(sin(x*y), Pow(Integer(2), -1))`. But instead, we have `Mul(Rational(1, 2), sin(x*y))`. Rational numbers are always combined into a single term in a multiplication, so that when we divide by 2, it is represented as multiplying by 1/2.

Finally, one last note. You may have noticed that the order we entered our expression and the order that it came out from `srepr` or in the graph were different. You may have also noticed this phenomenon earlier in the tutorial. For example

```
>>> 1 + x
x + 1
```

This because in SymPy, the arguments of the commutative operations `Add` and `Mul` are stored in an arbitrary (but consistent!) order, which is independent of the order inputted (if you're worried about noncommutative multiplication, don't be. In SymPy, you can create noncommutative Symbols using `Symbol('A', commutative=False)`, and the order of multiplication for noncommutative Symbols is kept the same as the input). Furthermore, as we shall see in the next section, the printing order and the order in which things are stored internally need not be the same either.

Quick Tip

The way an expression is represented internally and the way it is printed are often not the same.

Skip to content

In general, an important thing to keep in mind when working with SymPy expression trees is this: the internal representation of an expression and the way it is printed need not be the same. The same is true for the input form. If some expression manipulation algorithm is not working in the way you expected it to, chances are, the internal representation of the object is different from what you thought it was.

Recursing through an Expression Tree

Now that you know how expression trees work in SymPy, let's look at how to dig our way through an expression tree. Every object in SymPy has two very important attributes, `func`, and `args`.

func

`func` is the head of the object. For example, `(x*y).func` is `Mul`. Usually it is the same as the class of the object (though there are exceptions to this rule).

Two notes about `func`. First, the class of an object need not be the same as the one used to create it. For example

```
>>> expr = Add(x, x)
>>> expr.func
<class 'sympy.core.mul.Mul'>
```

We created `Add(x, x)`, so we might expect `expr.func` to be `Add`, but instead we got `Mul`. Why is that? Let's take a closer look at `expr`.

```
>>> expr
2*x
```

`Add(x, x)`, i.e., `x + x`, was automatically converted into `Mul(2, x)`, i.e., `2*x`, which is a `Mul`. SymPy classes make heavy use of the `__new__` class constructor, which, unlike `__init__`, allows a different class to be returned from the constructor.

Second, some classes are special-cased, usually for efficiency reasons [3].

```
>>> Integer(2).func
<class 'sympy.core.numbers.Integer'>
>>> Integer(0).func
<class 'sympy.core.numbers.Zero'>
>>> Integer(-1).func
<class 'sympy.core.numbers.NegativeOne'>
```

For the most part, these issues will not bother us. The special classes `Zero`, `One`, `NegativeOne`, classes of `Integer`, so as long as you use `isinstance`, it will not be an issue.

Skip to content

args

`args` are the top-level arguments of the object. `(x*y).args` would be `(x, y)`. Let's look at some examples

```
>>> expr = 3*y**2*x
>>> expr.func
<class 'sympy.core.mul.Mul'>
>>> expr.args
(3, x, y**2)
```

From this, we can see that `expr == Mul(3, y**2, x)`. In fact, we can see that we can completely reconstruct `expr` from its `func` and its `args`.

```
>>> expr.func(*expr.args)
3*x*y**2
>>> expr == expr.func(*expr.args)
True
```

Note that although we entered `3*y**2*x`, the `args` are `(3, x, y**2)`. In a `Mul`, the Rational coefficient will come first in the `args`, but other than that, the order of everything else follows no special pattern. To be sure, though, there is an order.

```
>>> expr = y**2*3*x
>>> expr.args
(3, x, y**2)
```

Mul's `args` are sorted, so that the same `Mul` will have the same `args`. But the sorting is based on some criteria designed to make the sorting unique and efficient that has no mathematical significance.

The `srepr` form of our `expr` is `Mul(3, x, Pow(y, 2))`. What if we want to get at the `args` of `Pow(y, 2)`. Notice that the `y**2` is in the third slot of `expr.args`, i.e., `expr.args[2]`.

```
>>> expr.args[2]
y**2
```

So to get the `args` of this, we call `expr.args[2].args`.

```
>>> expr.args[2].args
(y, 2)
```

Now what if we try to go deeper. What are the args of `y`. Or `2`. Let's see.

```
Skip to content
                .).args
()
```

They both have empty `args`. In SymPy, empty `args` signal that we have hit a leaf of the expression tree.

So there are two possibilities for a SymPy expression. Either it has empty `args`, in which case it is a leaf node in any expression tree, or it has `args`, in which case, it is a branch node of any expression tree. When it has `args`, it can be completely rebuilt from its `func` and its `args`. This is expressed in the key invariant.

> **Key Invariant**
>
> Every well-formed SymPy expression must either have empty `args` or satisfy `expr == expr.func(*expr.args)`.

(Recall that in Python if `a` is a tuple, then `f(*a)` means to call `f` with arguments from the elements of `a`, e.g., `f(*(1, 2, 3))` is the same as `f(1, 2, 3)`.)

This key invariant allows us to write simple algorithms that walk expression trees, change them, and rebuild them into new expressions.

Walking the Tree

With this knowledge, let's look at how we can recurse through an expression tree. The nested nature of `args` is a perfect fit for recursive functions. The base case will be empty `args`. Let's write a simple function that goes through an expression and prints all the `args` at each level.

```
>>> def pre(expr):
...     print(expr)
...     for arg in expr.args:
...         pre(arg)
```

See how nice it is that `()` signals leaves in the expression tree. We don't even have to write a base case for our recursion; it is handled automatically by the for loop.

Let's test our function.

```
>>> expr = x*y + 1
>>> pre(expr)
x*y + 1
1
x*y
x
y
```

Can you guess why we called our function `pre`? We just wrote a pre-order traversal function for our expression tree. See if you can write a post-order traversal function.

Skip to content

e so common in SymPy that the generator functions `preorder_traversal` and `sal` are provided to make such traversals easy. We could have also written our algorithm as

```
>>> for arg in preorder_traversal(expr):
...     print(arg)
x*y + 1
1
x*y
x
y
```

Prevent expression evaluation

There are generally two ways to prevent the evaluation, either pass an `evaluate=False` parameter while constructing the expression, or create an evaluation stopper by wrapping the expression with `UnevaluatedExpr`.

For example:

```
>>> from sympy import Add
>>> from sympy.abc import x, y, z
>>> x + x
2*x
>>> Add(x, x)
2*x
>>> Add(x, x, evaluate=False)
x + x
```

If you don't remember the class corresponding to the expression you want to build (operator overloading usually assumes `evaluate=True`), just use `sympify` and pass a string:

```
>>> from sympy import sympify
>>> sympify("x + x", evaluate=False)
x + x
```

Note that `evaluate=False` won't prevent future evaluation in later usages of the expression:

```
>>> expr = Add(x, x, evaluate=False)
>>> expr
x + x
>>> expr + x
3*x
```

That's why the class `UnevaluatedExpr` comes handy. `UnevaluatedExpr` is a method provided by SymPy which lets the user keep an expression unevaluated. By *unevaluated* it is meant that the value inside of it will not interact with the expressions outside of it to give simplified outputs. For example:

Skip to content

```
>>> from sympy import UnevaluatedExpr
>>> expr = x + UnevaluatedExpr(x)
>>> expr
x + x
>>> x + expr
2*x + x
```

The x remaining alone is the x wrapped by `UnevaluatedExpr`. To release it:

```
>>> (x + expr).doit()
3*x
```

Other examples:

```
>>> from sympy import *
>>> from sympy.abc import x, y, z
>>> uexpr = UnevaluatedExpr(S.One*5/7)*UnevaluatedExpr(S.One*3/4)
>>> uexpr
(5/7)*(3/4)
>>> x*UnevaluatedExpr(1/x)
x*1/x
```

A point to be noted is that `UnevaluatedExpr` cannot prevent the evaluation of an expression which is given as argument. For example:

```
>>> expr1 = UnevaluatedExpr(x + x)
>>> expr1
2*x
>>> expr2 = sympify('x + x', evaluate=False)
>>> expr2
x + x
```

Remember that `expr2` will be evaluated if included into another expression. Combine both of the methods to prevent both inside and outside evaluations:

```
>>> UnevaluatedExpr(sympify("x + x", evaluate=False)) + y
y + (x + x)
```

`UnevaluatedExpr` is supported by SymPy printers and can be used to print the result in different output forms. For example

```
>>> from sympy import latex
>>> uexpr = UnevaluatedExpr(S.One*5/7)*UnevaluatedExpr(S.One*3/4)
>>> print(latex(uexpr))
\frac{5}{7} \cdot \frac{3}{4}
```

Skip to content : the expression and get the evaluated LaTeX form, just use `.doit()`:

```
>>> print(latex(uexpr.doit()))
\frac{15}{28}
```

Footnotes

[1] We have been using `symbols` instead of `Symbol` because it automatically splits apart strings into multiple `Symbol`s. `symbols('x y z')` returns a tuple of three `Symbol`s. `Symbol('x y z')` returns a single `Symbol` called `x y z`.

[2] Technically, it is an internal function called `_sympify`, which differs from `sympify` in that it does not convert strings. `x + '2'` is not allowed.

[3] Classes like `One` and `Zero` are singletonized, meaning that only one object is ever created, no matter how many times the class is called. This is done for space efficiency, as these classes are very common. For example, `Zero` might occur very often in a sparse matrix represented densely. As we have seen, `NegativeOne` occurs any time we have `-x` or `1/x`. It is also done for speed efficiency because singletonized objects can be compared by `is`. The unique objects for each singletonized class can be accessed from the `S` object.

Copyright © 2024 SymPy Development Team
Made with Sphinx and @pradyunsg's Furo
Last updated on Sep 18, 2024